AttendIQ COA AI Learning Assistant

Conversation-Based Design, Architecture, Training, Evaluation, Monitoring, and Metric Tradeoff Document

Document purpose

This document consolidates the current conversation into a single professional engineering document. It covers the smart attendance and engagement product idea, system design, README scope, AI training strategy, data collection plan, monitoring dashboards, evaluation metrics, recommendation strategy, and AI metric tradeoff interpretation framework.

Field	Value
Project	AttendIQ COA Smart Attendance + Learning Intelligence Platform
Subject domain	Computer Organization and Architecture
Primary users	Students, faculty, academic admins
Scale assumption	150–200 students; 100–250 peak concurrent logins/check-ins
AI phase	OCR + COA Knowledge Graph + Flan-T5 LoRA + Recommendation Engine
Document style	AI engineer / system architect documentation

Table of Contents

1. Executive Summary

AttendIQ COA is a privacy-first classroom attendance, engagement, and AI learning assistant platform for Computer Organization and Architecture. The core idea is to make attendance a gateway to learning value rather than a compliance-only activity. Students attend classes because participation unlocks resources, feedback, and progress tracking. Faculty gain reliable attendance visibility, concept-check analytics, and weak-topic insights.

The AI learning assistant is a Phase 2 capability. It uses OCR to read handwritten answers or notes, maps extracted text to a COA concept taxonomy, uses a fine-tuned T5 model with LoRA to generate educational feedback, and recommends resources through a hybrid knowledge graph + rule + model pipeline.

Core principle

Knowledge Base First → Dataset Second → Fine-Tuning Third → Recommendations Fourth → Monitoring Everywhere.

2. Product Problem and Design Thinking

The original requirement was to design something that can make students come to COA classes. The proposed product reframes the problem from “take attendance” to “increase genuine classroom engagement.”

Stakeholder	Pain Point	Design Response
Student	COA feels abstract and difficult; attendance feels like a rule.	Unlock notes, quizzes, PYQs, revision resources, and AI feedback through participation.
Faculty	Manual attendance is time-consuming and proxy attendance is hard to control.	Use rotating QR, session time windows, optional proximity signal, and micro-quizzes.
Institution	Attendance data alone does not show learning outcomes.	Add analytics on weak topics, participation, and resource usage.

Design challenge: How might we increase genuine student attendance and active engagement in COA classes while respecting student privacy and operating under a free/open-source-first budget?

3. Core Platform Architecture

The MVP architecture can be implemented as a modular monolith with clean service boundaries. The system should be simple enough for low-cost deployment but modular enough to evolve into service separation later.

```
Student Mobile PWA
  → Authentication Service
  → Attendance API
  → Verification Engine
    → QR Token Service
    → Device Session Validator
    → Optional Classroom Signal Validator
  → PostgreSQL


Faculty Dashboard
```

→ Session Management API

→ Quiz API

→ Analytics API

→ Resource Management API

Phase 2 AI Service

→ OCR Engine

→ COA Concept Extractor

→ T5 LoRA Feedback Model

→ Recommendation Engine

Layer	Recommended Stack	Reason
Frontend	React / Next.js / PWA	Phone-first, easy deployment, works across devices.
Backend	FastAPI	Lightweight, Python-native, suitable for AI integration.
Database	PostgreSQL	Strong relational model for attendance, quizzes, users, rewards, logs.
Storage	Supabase Storage / S3-compatible storage	Stores PDFs, notes, AI uploads, resource files.
Monitoring	Prometheus + Grafana + MLflow + PostgreSQL	Covers real-time service metrics, training metrics, and product analytics.
AI	PaddleOCR + Flan-T5 LoRA + knowledge graph	Balances cost, accuracy, explainability, and maintainability.

4. Database and API Design Summary

The conversation defined a full README containing table schemas, endpoint request/response bodies, ER diagrams, UML diagrams, HLD, LLD, OpenAPI specification, and PostgreSQL DDL scripts. The major core tables are summarized below.

Table	Purpose
users	Stores students, faculty, and admins with roles and institutional identifiers.
courses	Stores COA course metadata and syllabus summary.
course_sections	Maps a course to faculty and academic term/batch.
enrollments	Maps students to course sections.
classrooms	Stores classroom metadata and optional proximity signals.
class_sessions	Stores each lecture/session with topic and status.
qr_tokens	Stores short-lived QR token hashes for attendance verification.
attendance_records	Stores verified/pending/rejected/manual attendance events.
quiz_questions	Stores COA concept-check questions.
session_quizzes	Maps questions to live class sessions.
quiz_responses	Stores student responses and correctness.

resources	Stores notes, PYQs, quizzes, practice files, and required reward tiers.
reward_policies	Defines bronze/silver/gold/platinum thresholds.
student_reward_status	Stores current attendance, participation, and reward tier.
ai_submissions	Stores OCR/AI feedback workflow outputs.
audit_logs	Tracks sensitive actions such as overrides and policy changes.

Major API groups include authentication APIs, course APIs, session APIs, QR/attendance APIs, quiz APIs, resource/reward APIs, analytics APIs, and Phase 2 AI submission APIs.

5. Production Repository Structure

```
attendiq-coa/
├── frontend/
│   ├── student-app/
│   ├── faculty-dashboard/
│   └── admin-dashboard/
├── backend/
│   ├── app/
│   │   ├── auth/
│   │   ├── users/
│   │   ├── courses/
│   │   ├── enrollments/
│   │   ├── classrooms/
│   │   ├── sessions/
│   │   ├── attendance/
│   │   ├── verification/
│   │   ├── quizzes/
│   │   ├── rewards/
│   │   ├── resources/
│   │   ├── analytics/
│   │   ├── audits/
│   │   └── ai/
├── ai-services/
│   ├── ocr-service/
│   ├── t5-lora-service/
│   ├── recommendation-engine/
│   └── weak-topic-detector/
├── database/
│   ├── schema/
│   ├── migrations/
```

```
│   └── seed/
├── infrastructure/
├── docs/
├── scripts/
├── README.md
├── docker-compose.yml
└── .env.example
```

6. AI Learning Assistant Objective

The AI assistant is not a generic chatbot. It is a COA-specific educational assistant. Its outputs should be grounded in a COA syllabus taxonomy, faculty-approved resources, and a controlled recommendation graph.

Capability	Description
Handwritten answer understanding	Uses OCR to extract text from uploaded answer images.
Concept coverage detection	Identifies which expected COA concepts are covered.
Missing concept detection	Finds important concepts absent from the answer.
Misconception detection	Flags risky or incorrect interpretations.
Educational feedback	Generates constructive feedback for improvement.
Resource recommendation	Maps missing concepts to notes, quizzes, videos, or assignments.
Faculty analytics	Aggregates weak topics and concept mastery across the class.

7. Data Collection and Resource Strategy

The model should not be trained directly on raw PDFs. PDFs and notes should be treated as source material used to build a structured knowledge base, a concept taxonomy, and instruction-style datasets.

Resource Type	What to Collect	Priority	Usage
University notes	COA lecture notes, digital notes, lab notes	High	Build foundational knowledge base.
Faculty resources	Slides, class notes, quizzes, assignments, rubrics	Highest	Align output to instructor and course style.
Question papers	Mid-sem, end-sem, PYQs, quizzes	High	Generate answer-evaluation examples.
Student answers	Excellent, good, average, weak, misconception answers	Highest for evaluation	Train concept coverage and feedback generation.
Public licensed resources	Open books, open lecture notes, tutorials	Medium	Fill gaps only when license permits.

Data governance

Only use institution-approved or legally reusable content. Anonymize student data. Do not use student submissions for training unless consent and policy allow it.

8. Preprocessing and OCR Pipeline

```
Raw PDFs / Images
  → File validation
  → Text extraction or OCR
  → Cleaning headers/footers/page numbers
  → Chunking by topic/subtopic/concept
  → Concept tagging
  → Knowledge graph ingestion
  → Dataset generation
```

Stage	Implementation Detail	Metric to Track
Text extraction	Use PyMuPDF/pdfplumber/Apache Tika for digital PDFs.	Extraction success rate.
OCR	Use PaddleOCR for handwritten/scanned notes; keep Tesseract as fallback.	OCR confidence, CER, WER.
Cleaning	Remove headers, footers, watermarks, page numbers, references.	Clean chunk ratio.
Chunking	Separate by chapter, subtopic, concept, formula, examples.	Average chunk length, topic coverage.
Concept tagging	Map chunks to COA taxonomy.	Tag accuracy, unmatched chunks.

9. Knowledge Base and Concept Taxonomy

```
COA
├── Functional Blocks
│   ├── CPU
│   ├── Memory
│   ├── I/O Subsystems
│   └── Control Unit
├── ISA
│   ├── Registers
│   ├── Instruction Cycle
│   ├── RTL
│   ├── Addressing Modes
│   └── Instruction Sets
├── Data Representation
│   ├── Signed Numbers
│   ├── Fixed Point
│   ├── Floating Point
│   └── Character Representation
├── Computer Arithmetic
```

```
│   ├── Ripple Carry Adder
│   ├── Carry Look Ahead Adder
│   ├── Booth Multiplier
│   ├── Carry Save Multiplier
│   └── Restoring/Non-Restoring Division
├── Control Unit
│   ├── Hardwired Control
│   └── Microprogrammed Control
├── Memory System
│   ├── Semiconductor Memory
│   ├── Memory Hierarchy
│   ├── Cache Mapping
│   ├── Replacement Policies
│   └── Write Policies
├── I/O
│   ├── Programmed I/O
│   ├── Interrupt Driven I/O
│   ├── DMA
│   └── Exceptions
└── Performance
    ├── Pipelining
    ├── Throughput
    ├── Speedup
    └── Pipeline Hazards
```

10. Dataset Generation and Synthetic Data Strategy

The training data should be instruction-style and task-specific rather than raw chapter text. This improves model controllability and allows meaningful evaluation.

Dataset Type	Example Task	Output
Concept explanation	Explain cache mapping.	Clear explanation with examples.
Concept coverage	Evaluate student answer against expected concepts.	Covered concepts, missing concepts.
Feedback generation	Give constructive feedback.	Student-friendly feedback.
Recommendation mapping	Given missing concepts, recommend resources.	Ranked resources.
Misconception detection	Detect incorrect explanation.	Misconception label and correction.

Synthetic data should generate excellent, good, average, weak, incomplete, and misconception answers for each topic. Synthetic examples must be reviewed or sampled by faculty before being used as high-confidence training data.

11. Model Selection and LoRA Training Strategy

Option	Pros	Cons	Decision
Flan-T5 Base	Fast and cheap.	Lower feedback quality.	Good baseline.
Flan-T5 Large	Better educational feedback and still manageable.	Higher latency/cost than base.	Recommended MVP candidate.
Llama-class larger models	Strong generation.	Higher cost, latency, and potential hallucination.	Use only if T5 quality is insufficient.

```
Recommended LoRA configuration:
rank: 16
alpha: 32
dropout: 0.05
learning_rate: 2e-5
batch_size: 8
epochs: 3
split: train 80%, validation 10%, test 10%
```

12. Validation and Evaluation Metrics

Metric Category	Metrics	Why It Matters
OCR	OCR confidence, CER, WER, OCR failure rate	Bad OCR corrupts all downstream feedback.
Model quality	Coverage accuracy, missing concept precision/recall, hallucination rate, JSON validity	Measures correctness and reliability of feedback.
Recommendation	CTR, completion rate, post-resource quiz improvement, faculty override rate	Measures whether suggestions actually help.
User impact	Student helpfulness rating, faculty rating, weak-topic improvements	Measures educational usefulness.
Engineering	Latency, throughput, error rate, queue depth, CPU/GPU usage	Measures service health and scalability.

13. Recommendation Engine Strategy

Recommendations should not depend only on model generation. Use a hybrid approach: concept extraction + knowledge graph lookup + ranking formula + faculty overrides.

```
Student Answer
  → Concept Coverage Detection
  → Missing Concepts
  → Knowledge Graph Lookup
  → Resource Ranking
  → Recommendation
  → Resource Usage Logging
```

```
  → Post-Recommendation Quiz Impact
resource_score =
0.40 × concept_match_score
+ 0.25 × difficulty_match_score
+ 0.20 × historical_success_score
+ 0.10 × faculty_priority_score
+ 0.05 × freshness_score
```

14. Monitoring Dashboard Implementation

Monitoring should be split into focused dashboards instead of one overloaded dashboard. The recommended stack is Prometheus + Grafana for runtime metrics, MLflow for training and experiment tracking, PostgreSQL for product/recommendation events, and Evidently-style batch jobs for AI quality reports.

Dashboard	Panels
AI Service Health	Requests/min, error rate, P50/P95/P99 latency, OCR latency, model inference latency, queue depth, CPU/GPU memory.
OCR Quality	Average OCR confidence, low-confidence upload rate, OCR failure rate, CER, WER, reupload rate.
Model Quality	Concept coverage accuracy, missing precision/recall, hallucination rate, JSON validity, faculty score.
Recommendation Quality	CTR, completion rate, quiz improvement, recommendation coverage, faculty override rate.
Faculty Learning Analytics	Weakest topics, class mastery heatmap, progression over time, resource usage by topic.

```
Example FastAPI metric names:
ai_requests_total
ai_request_latency_seconds
ocr_confidence_score
ocr_processing_time_seconds
model_inference_latency_seconds
hallucination_rate
json_validity_rate
recommendation_ctr
post_resource_quiz_improvement
```

15. AI Metric Tradeoff Interpretation Framework

AI systems should not be optimized for one metric. Improving accuracy can increase latency, improving recall can reduce precision, and improving model size can increase cost. The correct strategy is to optimize the metric portfolio according to risk priority.

Metric priority order

Safety → Accuracy → Educational Value → Latency → Cost

Tradeoff	Interpretation	Recommended Decision
OCR accuracy vs speed	Higher OCR accuracy may increase latency, but bad OCR damages all downstream outputs.	Prefer better OCR unless latency becomes unusable.
Accuracy vs hallucination	A model with slightly lower accuracy but much lower hallucination is safer.	Choose lower hallucination for educational trust.
Precision vs recall	High precision avoids false missing-concept claims; high recall catches more gaps.	Balance both around 80–85% for MVP.
Latency vs quality	A larger model may improve feedback but slow the student experience.	Do not double latency for small accuracy gains.
Recommendation precision vs diversity	Precise recommendations may become repetitive; diverse recommendations may be less relevant.	Use mostly precision with controlled diversity.
Cost vs model quality	Very large models may not justify marginal quality gains.	Start with Flan-T5 Large + LoRA; upgrade only if metrics prove need.

16. Release Gates and Decision Rules

Metric	Target / Gate	Decision If Failed
Concept coverage accuracy	>= 85%	Add labeled concept examples and retrain.
Missing concept precision	>= 80%	Reduce over-flagging; add negative examples.
Missing concept recall	>= 80%	Add weak/incomplete answer examples.
Hallucination rate	<= 5%	Block release; constrain output to taxonomy.
JSON validity rate	>= 98%	Use schema validation and retry/constrained decoding.
OCR confidence	>= 90% average for accepted images	Improve preprocessing or ask re-upload.
Faculty usefulness rating	>= 4/5	Improve feedback style and rubric alignment.
P95 latency	<= 5 seconds for synchronous feedback	Use async processing, smaller model, quantization, or caching.

17. MLOps Pipeline and Deployment

Data Collection

→ Preprocessing

→ Knowledge Base

→ Dataset Generator

→ Train/Validation/Test Split

→ LoRA Training

→ MLflow Tracking

→ Evaluation and Faculty Review

→ Model Registry

→ Inference API

→ Monitoring Dashboards

→ Feedback Loop and Retraining

For deployment, use a lightweight inference service for MVP. Heavy AI tasks can run asynchronously through a queue. The core attendance platform should remain independent from the AI service so that AI failures do not break attendance or resource access.

18. Final Engineering Recommendation

The strongest architecture is a modular education platform where attendance, engagement, resources, and AI feedback are connected but separable. Start with the attendance and quiz MVP, then add AI feedback after a knowledge base and evaluation pipeline are ready.

Final recommended path:

1. Build COA concept taxonomy.
2. Collect approved notes, faculty resources, PYQs, quizzes, and anonymized answers.
3. Extract and clean text using PDF extraction and OCR.
4. Build a concept/resource knowledge graph.
5. Generate instruction datasets.
6. Fine-tune Flan-T5 Large with LoRA.
7. Evaluate using concept coverage, missing precision/recall, hallucination rate, JSON validity, faculty rating, and recommendation impact.
8. Deploy with monitoring dashboards.
9. Improve using feedback and metric-driven tradeoff decisions.

Final takeaway

Knowledge Base First → Dataset Second → Fine-Tuning Third → Recommendations Fourth → Monitoring Everywhere.

Appendix A: Sources Referenced During Conversation

The conversation used publicly available documentation and prior generated project artifacts as grounding for parts of the architecture and monitoring discussion. The document itself is a synthesized engineering design based on the conversation, not a direct quotation of those sources.

Source Category	Examples Used in Conversation
COA educational sources	MRCET COA notes, IIT Delhi architecture book, Purdue architecture notes, EC8552 notes.
Monitoring tools	Grafana + Prometheus documentation, MLflow tracking documentation, Evidently AI documentation.
Generated project artifacts	AttendIQ pitch deck, prior README design documents, HLD/LLD/OpenAPI/DDL additions.